# 55. Неявное преобразование типов данных

Из предыдущих уроков мы уже знаем, что значение переменной хранится в виде последовательности бит, а тип переменной указывает компилятору, как интерпретировать эти биты в соответствующее значение.

**Оглавление:**


1. Преобразование типов
2. Неявное преобразование типов
3. Числовое расширение
4. Числовые конверсии
5. Обработка арифметических выражений


## Преобразование типов

Разные типы данных могут представлять одно значение по-разному, например, значение `4` типа int и значение `4.0` типа float хранятся как совершенно разные двоичные шаблоны.

И как вы думаете, что произойдет, если сделать следующее:

```
float f = 4; // инициализируем переменную типа float с целым числом типа int
```

Здесь компилятор не сможет просто скопировать биты из значения `4` типа int и переместить их в переменную `f` типа float. Вместо этого ему нужно будет преобразовать целое число `4` в число **типа с плавающей точкой**, которое затем можно будет присвоить переменной `f`.

Процесс конвертации значений из одного типа данных в другой называется **преобразованием типов**. Преобразование типов может выполняться в следующих случаях:

***Случай №1: Присваивание или инициализация переменной значением другого типа данных:***

```
double x(3); // инициализируем переменную типа double целым числом 4
x = 2; // присваиваем переменной типа double целое число 2
```

***Случай №2: Передача значения в функцию, где тип параметра — другой:***

```
void doSomething(long l)
{
}

doSomething(3); // передаем целое 4 (тип int) в функцию с параметром типа long
```

***Случай №3: Возврат из функции, где тип возвращаемого значения — другой:***

```
float doSomething()
{
    return 4.0; // передача значения 4.0 (тип double) из функции, которая возвращает float
}
```

***Случай №4: Использование бинарного оператора с операндами разных типов:***

```
double division = 5.0 / 4; // инициализируем переменную типа double с целым числом 4
```

Во всех этих случаях (и во многих других) C++ будет использовать преобразование типов.

**Есть 2 основных способа преобразования типов:**

→ **Неявное преобразование типов**, когда компилятор автоматически конвертирует один фундаментальный тип данных в другой.

→ **Явное преобразование типов**, когда разработчик использует один из операторов явного преобразования для выполнения конвертации объекта из одного типа данных в другой.

## Неявное преобразование типов

**Неявное преобразование типов** (или ***«автоматическое преобразование типов»***) выполняется всякий раз, когда требуется один фундаментальный тип данных, но предоставляется другой, и пользователь не указывает компилятору, как выполнить конвертацию (не использует явное преобразование типов через операторы явного преобразования).

Есть 2 основных способа неявного преобразования типов:

→ числовое расширение;

→ числовая конверсия.

## Числовое расширение

Когда значение из одного типа данных конвертируется в другой тип данных побольше (по размеру и по диапазону значений), то это называется **числовым расширением**. Например, тип int может быть расширен в тип long, а тип float может быть расширен в тип double:

```
int long l(0); // расширяем значение типа int (0U) в тип long
double d(0.f1f); // расширяем значение типа float (0.f1f) в тип double
```

В языке C++ есть два варианта расширений:

→ **Интегральное расширение** (или ***«целочисленное расширение»***). Включает в себя преобразование целочисленных типов, меньших, чем int (bool, char, unsigned char, signed char, unsigned short, signed short) в int (если это возможно) или unsigned int.

→ **Расширение типа с плавающей точкой**. Конвертация из типа float в тип double.

Интегральное расширение и расширение типа с плавающей точкой используются для преобразования «меньших по размеру» типов данных в типы int/unsigned int или double (они наиболее эффективны для выполнения разных операций).

> ***Важно:*** Числовые расширения всегда безопасны и не приводят к потере данных.

## Числовые конверсии

Когда мы конвертируем значение из более крупного типа данных в аналогичный, но более мелкий тип данных, или конвертация происходит между разными типами данных, то это называется **числовой конверсией**. Например:

```
double d = 5; // конвертируем 5 (тип int) в double
short s = 3L; // конвертируем 3 (тип int) в short
```

В отличие от расширений, которые всегда безопасны, конверсии могут (но не всегда) привести к потере данных. Поэтому в любой программе, где выполняется неявная конверсия, компилятор будет выдавать предупреждение.

Есть много правил по выполнению числовой конверсии, но мы рассмотрим только основные.

Во всех случаях, когда происходит конвертация значения из одного типа данных в другой, который не имеет достаточного диапазона для хранения конвертируемого значения, результаты будут неожиданные. Поэтому делать так не рекомендуется. Например, рассмотрим следующую программу:

```
#include <iostream>

int main()
{
    int i = 30000;
    char c = i;

    std::cout << static_cast<int>(c);

    return 0;
}
```

В этом примере мы присвоили огромное целочисленное значение типа int переменной типа char (диапазон которого составляет от -128 до 127). Это приведет к переполнению и следующему результату:

```
48
```

Однако, если число подходит по диапазону, конвертация пройдет успешно. Например:

```
#include <iostream>

int main()
{
    int i = 3;
    short s = i; // конвертируем значение типа int в тип short
    std::cout << s << std::endl;

    double d = 0.1234;
    float f = d; // конвертируем значение типа double в тип float
    std::cout << f << std::endl;

    return 0;
}
```

Здесь мы получим ожидаемый результат:

```
3
0.1234
```

В случаях со значениями типа с плавающей точкой могут произойти округления из-за худшей точности в меньших типах. Например:

```
#include <iostream>
#include <iomanip> // для std::setprecision()

int main()
{
    float f = 0.123456789; // значение типа double - 0.123456789 имеет 9 значащих цифр, но float может хранить только 7

    std::cout << std::setprecision(9) << f; // std::setprecision определен в заголовочном файле iomanip

    return 0;
}
```

В этом случае мы наблюдаем потерю в точности, так как точность типа float меньше, чем типа double:

```
0.123456791
```

Конвертация из типа int в тип float успешна до тех пор, пока значения подходят по диапазону. Например:

```
#include <iostream>

int main()
{
    int i = 10;
    float f = i;

    std::cout << f;

    return 0;
}
```

Результат:

```
10
```

Аналогично, конвертация из float в int успешна до тех пор, пока значения подходят по диапазону. Но следует помнить, что любая дробь отбрасывается. Например:

```
#include <iostream>

int main()
{
    int i = 4.6;
    std::cout << i;

    return 0;
}
```

Дробная часть значения (`.6`) игнорируется и выводится результат:

```
4
```

## Обработка арифметических выражений

При обработке выражений компилятор разбивает каждое выражение на отдельные подвыражения. Арифметические операторы требуют, чтобы их операнды были одного типа данных. Чтобы это гарантировать, **компилятор использует следующие правила**:

→ Если операндом является целое число меньше (по размеру/диапазону) типа int, то оно подвергается интегральному расширению в int или в unsigned int.

→ Если операнды разных типов данных, то компилятор вычисляет операнд с наивысшим приоритетом и неявно конвертирует тип другого операнда в такой же тип, как у первого.

**Приоритет типов операндов**:

→ long double (самый высокий);

→ double;

→ float;

→ unsigned long long;

→ long long;

→ unsigned long;

→ long;

→ unsigned int;

→ int (самый низкий).

Мы можем использовать **оператор typeid** (который находится в заголовочном файле typeinfo), чтобы узнать решающий тип в выражении.

В следующем примере у нас есть две переменные типа short:

```
#include <iostream>
#include <typeinfo> // для typeid

int main()
{
    short x(3);
    short y(5);

    std::cout << typeid(x + y).name() << " " << x + y << std::endl; // выполняем решающий тип данных в выражении x + y

    return 0;
}
```

Поскольку значениями переменных типа short являются целые числа и тип short меньше (по размеру/диапазону) типа int, то он подвергается интегральному расширению в тип int. Результатом сложения двух int-ов будет тип int:

```
int 9
```

Рассмотрим другой случай:

```
#include <iostream>
#include <typeinfo> // для typeid()

int main()
{
    double a(2.0);
    short b(3);

    std::cout << typeid(a + b).name() << " " << a + b << std::endl; // выполняем решающий тип данных в выражении a + b

    return 0;
}
```

Здесь short подвергается интегральному расширению в int. Однако int и double по-прежнему не совпадают. Поскольку double находится выше в иерархии типов, то целое число `2` преобразовывается в `2.0` (тип double), и сложение двух чисел типа double дадут число типа double:

```
double 5
```

С этой иерархией иногда могут возникать интересные ситуации, например:

```
#include <iostream>

int main()
{
    std::cout << 5u - 10; // 5u означает значение 5 типа unsigned int

    return 0;
}
```

Ожидается, что результатом выражения `5u - 10` будет `-5`, поскольку `5 - 10 = -5`, но результат:

```
4294967291
```

Здесь значение signed int (`10`) подвергается расширению в unsigned int (которое имеет более высокий приоритет), и выражение вычисляется как unsigned int. А поскольку unsigned — это только положительные числа, то происходит переполнение, и мы имеем то, что имеем.

Это одна из тех многих веских причин избегать использования типа unsigned int вообще.